

Kali Linux Raspberry Pi 5 Setup Guide with Persistence and VM Support

Table of Contents

1. [Prerequisites](#)
 2. [Installing Kali Linux on Raspberry Pi 5](#)
 3. [Setting Up Persistence](#)
 4. [Configuring VM Support](#)
 5. [Post-Installation Configuration](#)
 6. [Troubleshooting](#)
 7. [Advanced Configuration](#)
-

Prerequisites

Hardware Requirements

- **Raspberry Pi 5:** 4GB or 8GB RAM model (8GB recommended for VM support)
- **MicroSD Card:** Class 10, 64GB minimum (128GB+ recommended)
- **USB Storage:** High-speed USB 3.0 drive for persistence (optional but recommended)
- **Power Supply:** Official Raspberry Pi 5 USB-C power adapter (5V/5A)
- **Cooling:** Active cooling fan or heatsink (essential for sustained performance)
- **Network:** Ethernet cable or compatible WiFi adapter

Additional Hardware (Optional)

- **USB-to-Ethernet Adapter:** For additional network interfaces
- **USB Hub:** For connecting multiple devices
- **GPIO Breakout Board:** For hardware testing
- **HDMI Cable:** For display output
- **Keyboard and Mouse:** For initial setup

Software Requirements

- **Raspberry Pi Imager:** Latest version from official Raspberry Pi website
- **Kali Linux ARM64 Image:** Official Kali Linux image for Raspberry Pi

- **SSH Client:** For remote access (PuTTY, OpenSSH)
- **VNC Viewer:** For remote desktop access

Downloads

1. **Raspberry Pi Imager:** Download from <https://www.raspberrypi.com/software/>
 2. **Kali Linux ARM64:** Download from <https://www.kali.org/get-kali/>
 3. **Balena Etcher:** Alternative flashing tool from <https://www.balena.io/etcher/>
-

Installing Kali Linux on Raspberry Pi 5

Step 1: Prepare the MicroSD Card

Using Raspberry Pi Imager (Recommended):

1. **Install Raspberry Pi Imager:**
 - Download and install from official website
 - Launch the application
2. **Configure Imager Settings:**
 - Click "CHOOSE OS"
 - Select "Use custom image"
 - Browse and select Kali Linux ARM64 image
3. **Advanced Configuration:**
 - Click gear icon for advanced options
 - **Enable SSH:** Check "Enable SSH" and set password
 - **Configure WiFi:** Enter network credentials
 - **Set Username:** Create user account (default: kali/kali)
 - **Configure Locale:** Set timezone and keyboard layout
4. **Write to SD Card:**
 - Insert MicroSD card
 - Click "CHOOSE STORAGE" and select SD card
 - Click "WRITE" and wait for completion

Using Balena Etcher:

1. **Launch Balena Etcher**

2. **Select Image:** Choose Kali Linux ARM64 image
3. **Select Target:** Choose MicroSD card
4. **Flash:** Click "Flash!" and wait for completion

Step 2: Initial Boot and Setup

1. Insert SD Card and Boot:

- Insert MicroSD card into Raspberry Pi 5
- Connect power supply
- Connect HDMI, keyboard, and mouse
- Power on the device

2. Initial Login:

- Default credentials: `kali` / `kali`
- You'll be prompted to change password on first login

3. Enable SSH (if not done via Imager):

```
bash
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

4. Update System:

```
bash
```

```
sudo apt update && sudo apt upgrade -y
```

Step 3: Configure Network Settings

WiFi Configuration:

1. Using Command Line:

```
bash
```

```
# Scan for networks
```

```
sudo iwlist wlan0 scan | grep ESSID
```

```
# Configure network
```

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

```
# Add network configuration:
```

```
network={  
    ssid="YourNetworkName"  
    psk="YourPassword"  
}
```

```
# Restart networking
```

```
sudo systemctl restart networking
```

2. Using NetworkManager:

```
bash
```

```
# List available networks
```

```
nmcli dev wifi list
```

```
# Connect to network
```

```
nmcli dev wifi connect "NetworkName" password "YourPassword"
```

Ethernet Configuration:

1. Static IP Configuration:

```
bash
```

```
sudo nano /etc/dhcpd.conf
```

```
# Add static IP configuration:
```

```
interface eth0
```

```
static ip_address=192.168.1.100/24
```

```
static routers=192.168.1.1
```

```
static domain_name_servers=8.8.8.8 8.8.4.4
```

Setting Up Persistence

Step 4: Configure SD Card Persistence

Expand Root Partition:

1. Check current partition layout:

```
bash
```

```
df -h
```

```
lsblk
```

2. Expand root partition:

```
bash
```

```
sudo raspi-config
```

```
# Select "Advanced Options" → "Expand Filesystem"
```

```
# Or manually with parted:
```

```
sudo parted /dev/mmcblk0
```

```
(parted) print
```

```
(parted) resizepart 2 100%
```

```
(parted) quit
```

```
sudo resize2fs /dev/mmcblk0p2
```

Step 5: Configure USB Persistence

Prepare USB Drive:

1. Connect USB drive and identify device:

```
bash
```

```
lsblk
```

```
# Example output: /dev/sda (your USB drive)
```

2. Create persistence partition:

```
bash
```

```
# Unmount if mounted
```

```
sudo umount /dev/sda*
```

```
# Create new partition table
```

```
sudo parted /dev/sda
```

```
(parted) mklabel gpt
```

```
(parted) mkpart primary ext4 0% 100%
```

```
(parted) quit
```

```
# Format partition
```

```
sudo mkfs.ext4 -L persistence /dev/sda1
```

3. Configure persistence:

```
bash
```

```
# Create mount point
```

```
sudo mkdir -p /mnt/persistence
```

```
# Mount USB drive
```

```
sudo mount /dev/sda1 /mnt/persistence
```

```
# Create persistence directories
```

```
sudo mkdir -p /mnt/persistence/home
```

```
sudo mkdir -p /mnt/persistence/var
```

```
sudo mkdir -p /mnt/persistence/opt
```

```
# Set up auto-mount
```

```
echo "/dev/sda1 /mnt/persistence ext4 defaults,noatime 0 2" | sudo tee -a /etc/fstab
```

4. Create symbolic links for persistence:

```
bash
```

```
# Backup original directories
```

```
sudo cp -r /home /home.backup
```

```
sudo cp -r /var /var.backup
```

```
sudo cp -r /opt /opt.backup
```

```
# Create symbolic links
```

```
sudo rm -rf /home /var/log /opt/tools
```

```
sudo ln -s /mnt/persistence/home /home
```

```
sudo ln -s /mnt/persistence/var /var
```

```
sudo ln -s /mnt/persistence/opt /opt
```

Configuring VM Support

Step 6: Enable Hardware Virtualization

Check Virtualization Support:

1. Verify ARM virtualization extensions:

```
bash
```

```
# Check for virtualization support
```

```
grep -E 'Features.*virt' /proc/cpuinfo
```

```
# Install virtualization tools
```

```
sudo apt install qemu-system-arm qemu-utils
```

2. Enable KVM support:

```
bash
```

```
# Check KVM availability
```

```
ls -la /dev/kvm
```

```
# Install KVM packages
```

```
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils
```

Step 7: Install Container Runtime

Docker Installation:

1. Install Docker:

```
bash
```

```
# Update package index
```

```
sudo apt update
```

```
# Install Docker dependencies
```

```
sudo apt install apt-transport-https ca-certificates curl gnupg lsb-release
```

```
# Add Docker's official GPG key
```

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-arc
```

```
# Add Docker repository
```

```
echo "deb [arch=arm64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com
```

```
# Install Docker
```

```
sudo apt update
```

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

```
# Add user to docker group
```

```
sudo usermod -a -G docker $USER
```

2. Configure Docker for ARM64:

```
bash
```

```
# Enable Docker service
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
# Test Docker installation
```

```
docker run hello-world
```

Podman Installation (Alternative):

1. Install Podman:

```
bash
```

```
sudo apt install podman
```

```
# Configure rootless containers
```

```
echo "kali:10000:65536" | sudo tee -a /etc/subuid
```

```
echo "kali:10000:65536" | sudo tee -a /etc/subgid
```


Step 8: Install Lightweight VM Solutions

QEMU/KVM Setup:

1. Install QEMU packages:

```
bash
```

```
sudo apt install qemu-system-arm qemu-system-aarch64 qemu-utils  
sudo apt install libvirt-daemon-system libvirt-clients virt-manager
```

2. Configure libvirt:

```
bash
```

```
# Add user to libvirt group  
sudo usermod -a -G libvirt $USER
```

```
# Start libvirt service  
sudo systemctl enable libvirtd  
sudo systemctl start libvirtd
```

```
# Configure default network  
sudo virsh net-autostart default  
sudo virsh net-start default
```

3. Create ARM64 VM:

```
bash
```

```
# Download ARM64 operating system ISO  
# Example: Ubuntu Server ARM64
```

```
# Create VM with virt-install  
sudo virt-install \  
--name ubuntu-arm64 \  
--ram 2048 \  
--disk path=/var/lib/libvirt/images/ubuntu-arm64.qcow2,size=20 \  
--vcpus 2 \  
--os-type linux \  
--os-variant ubuntu20.04 \  
--network bridge=virbr0 \  
--graphics none \  
--console pty,target_type=serial \  
--cdrom /path/to/ubuntu-server-arm64.iso
```

Post-Installation Configuration

Step 9: Essential Tools and Updates

Install Penetration Testing Tools:

1. Update Kali repositories:

```
bash
```

```
sudo apt update && sudo apt upgrade -y
```

2. Install essential tools:

```
bash
```

```
# Network scanning tools
```

```
sudo apt install nmap masscan zmap
```

```
# Web application testing
```

```
sudo apt install burpsuite dirb gobuster nikto
```

```
# Wireless security tools
```

```
sudo apt install aircrack-ng reaver hostapd
```

```
# Forensics tools
```

```
sudo apt install autopsy sleuthkit volatility3
```

```
# Development tools
```

```
sudo apt install build-essential git vim python3-pip
```

3. Install ARM64-specific tools:

```
bash
```

```
# Install tools compiled for ARM64
```

```
sudo apt install metasploit-framework
```

```
sudo apt install sqlmap
```

```
sudo apt install john hashcat
```

Configure GPIO and Hardware Interfaces:

1. Enable GPIO interface:

```
bash
```

```
# Install GPIO libraries
```

```
sudo apt install python3-gpiozero python3-rpi.gpio
```

```
# Enable SPI and I2C
```

```
sudo raspi-config
```

```
# Select "Interface Options" → Enable SPI and I2C
```

2. Install hardware testing tools:

```
bash
```

```
# Install hardware analysis tools
```

```
sudo apt install minicom screen picocom
```

```
sudo apt install i2c-tools spi-tools
```

Step 10: Configure Remote Access

SSH Configuration:

1. Secure SSH setup:

```
bash
```

```
# Generate SSH key pair
```

```
ssh-keygen -t ed25519 -C "pi5-kali"
```

```
# Configure SSH daemon
```

```
sudo nano /etc/ssh/sshd_config
```

```
# Recommended settings:
```

```
Port 2222
```

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

```
# Restart SSH service
```

```
sudo systemctl restart ssh
```

VNC Configuration:

1. Install VNC server:

```
bash
```

```
sudo apt install tigervnc-standalone-server
```

```
# Start VNC server
```

```
vncserver :1
```

```
# Configure VNC startup
```

```
nano ~/.vnc/xstartup
```

```
# Add content:
```

```
#!/bin/bash
```

```
xrdb $HOME/.Xresources
```

```
startxfce4 &
```

2. Configure VNC service:

```
bash
```

```
# Create systemd service
```

```
sudo nano /etc/systemd/system/vncserver@.service
```

```
# Add service configuration:
```

```
[Unit]
```

```
Description=Start TightVNC server at startup
```

```
After=syslog.target network.target
```

```
[Service]
```

```
Type=forking
```

```
User=kali
```

```
Group=kali
```

```
WorkingDirectory=/home/kali
```

```
ExecStart=/usr/bin/vncserver -depth 24 -geometry 1920x1080 :%i
```

```
ExecStop=/usr/bin/vncserver -kill :%i
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
# Enable service
```

```
sudo systemctl enable vncserver@1.service
```

```
sudo systemctl start vncserver@1.service
```

Step 11: Performance Optimization

System Performance:

1. Configure GPU memory split:

bash

`sudo raspi-config`

Select "Advanced Options" → "Memory Split" → Set to 64MB

2. Optimize boot performance:

bash

Edit boot configuration

`sudo nano /boot/config.txt`

Add performance optimizations:

Overclock settings (use with good cooling)

`over_voltage=2`

`arm_freq=2200`

`gpu_freq=750`

Disable unnecessary services

`sudo systemctl disable bluetooth`

`sudo systemctl disable hciuart`

3. Configure swap file:

bash

Create swap file on USB drive

`sudo fallocate -l 4G /mnt/persistence/swapfile`

`sudo chmod 600 /mnt/persistence/swapfile`

`sudo mkswap /mnt/persistence/swapfile`

`sudo swapon /mnt/persistence/swapfile`

Make permanent

`echo "/mnt/persistence/swapfile none swap sw 0 0" | sudo tee -a /etc/fstab`

Troubleshooting

Common Issues and Solutions

Boot Issues:

- **Problem:** Pi 5 not booting from SD card
- **Solution:**

- Check SD card compatibility (Class 10 minimum)
- Verify image integrity with checksum
- Try different SD card
- Check power supply (5V/5A required)

Network Issues:

- **Problem:** WiFi not connecting
- **Solution:**
 - Check WiFi credentials in `/etc/wpa_supplicant/wpa_supplicant.conf`
 - Verify network compatibility (2.4GHz vs 5GHz)
 - Update WiFi firmware: `sudo apt update && sudo apt upgrade`
 - Check regional settings: `sudo raspi-config`

Performance Issues:

- **Problem:** System running slowly
- **Solution:**
 - Check temperature: `vcgencmd measure_temp`
 - Install active cooling
 - Reduce GPU memory split
 - Optimize swap settings
 - Use faster storage (USB 3.0)

VM Issues:

- **Problem:** VMs not starting or poor performance
- **Solution:**
 - Verify KVM support: `ls -la /dev/kvm`
 - Check available memory: `free -h`
 - Allocate appropriate resources
 - Use lightweight distributions in VMs

Hardware Interface Issues:

- **Problem:** GPIO/SPI/I2C not working
- **Solution:**

- Enable interfaces in `raspi-config`
 - Check device permissions
 - Verify wiring connections
 - Install appropriate libraries
-

Advanced Configuration

Step 12: Custom Tool Installation

Install Additional Security Tools:

1. Install from source:

```
bash
```

```
# Install Go for ARM64 tools
```

```
wget https://go.dev/dl/go1.21.linux-arm64.tar.gz
```

```
sudo tar -xzf go1.21.linux-arm64.tar.gz -C /usr/local
```

```
export PATH=$PATH:/usr/local/go/bin
```

```
# Install custom tools
```

```
go install github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
```

```
go install github.com/projectdiscovery/httpx/cmd/httpx@latest
```

2. Install Python tools:

```
bash
```

```
# Install Python security tools
```

```
pip3 install impacket
```

```
pip3 install bloodhound
```

```
pip3 install crackmapexec
```

Hardware-Specific Tools:

1. Install radio frequency tools:

```
bash
```

```
# Install SDR tools
```

```
sudo apt install gqrx-sdr rtl-sdr
```

```
sudo apt install gnuradio
```

```
# Install HackRF tools
```

```
sudo apt install hackrf
```

2. Install automotive tools:

```
bash
```

```
# Install CAN bus tools
```

```
sudo apt install can-utils
```

```
sudo apt install python3-can
```

Step 13: Automation and Monitoring

Create Monitoring Scripts:

1. System monitoring script:

```
bash
```

```
#!/bin/bash
```

```
# ~/scripts/monitor.sh
```

```
# System temperature
```

```
temp=$(vcgencmd measure_temp | cut -d= -f2)
```

```
echo "Temperature: $temp"
```

```
# System load
```

```
uptime
```

```
# Memory usage
```

```
free -h
```

```
# Disk usage
```

```
df -h
```

```
# Network status
```

```
ip addr show wlan0 | grep inet
```

2. Automated backup script:


```
bash
```

```
#!/bin/bash
```

```
# ~/scripts/backup.sh
```

```
# Backup critical configurations
```

```
backup_dir="/mnt/persistence/backups/$(date +%Y%m%d)"
```

```
mkdir -p "$backup_dir"
```

```
# Backup configurations
```

```
cp -r ~/.ssh "$backup_dir/"
```

```
cp -r ~/.config "$backup_dir/"
```

```
cp /etc/fstab "$backup_dir/"
```

```
# Backup custom tools
```

```
tar -czf "$backup_dir/custom-tools.tar.gz" /opt/custom-tools/
```

Service Management:

1. Create custom service:

```
bash
```

```
# Create service file
```

```
sudo nano /etc/systemd/system/kali-monitor.service
```

```
[Unit]
```

```
Description=Kali System Monitor
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=kali
```

```
ExecStart=/home/kali/scripts/monitor.sh
```

```
Restart=always
```

```
RestartSec=60
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
# Enable service
```

```
sudo systemctl enable kali-monitor.service
```

```
sudo systemctl start kali-monitor.service
```

Step 14: Network Configuration

Access Point Setup:

1. Configure hostapd:

```
bash
```

```
sudo apt install hostapd dnsmasq
```

```
# Configure hostapd
```

```
sudo nano /etc/hostapd/hostapd.conf
```

```
# Add configuration:
```

```
interface=wlan1
```

```
driver=nl80211
```

```
ssid=Kali-Pi5-AP
```

```
hw_mode=g
```

```
channel=7
```

```
wmm_enabled=0
```

```
macaddr_acl=0
```

```
auth_algs=1
```

```
ignore_broadcast_ssid=0
```

```
wpa=2
```

```
wpa_passphrase=YourPassword
```

```
wpa_key_mgmt=WPA-PSK
```

```
wpa_pairwise=TKIP
```

```
rsn_pairwise=CCMP
```

2. Configure DHCP:

```
bash
```

```
# Configure dnsmasq
```

```
sudo nano /etc/dnsmasq.conf
```

```
# Add DHCP configuration:
```

```
interface=wlan1
```

```
dhcp-range=192.168.4.2,192.168.4.20,255.255.255.0,24h
```

Bridge Configuration:

1. Create network bridge:

```
bash
```

```
sudo apt install bridge-utils
```

```
# Create bridge
```

```
sudo nano /etc/systemd/network/br0.netdev
```

```
[NetDev]
```

```
Name=br0
```

```
Kind=bridge
```

```
# Configure bridge network
```

```
sudo nano /etc/systemd/network/br0.network
```

```
[Match]
```

```
Name=br0
```

```
[Network]
```

```
DHCP=yes
```

```
IPMasquerade=yes
```

Performance Optimization

Hardware Optimization:

Cooling Solutions:

- **Active Cooling:** Install PWM fan with temperature control
- **Passive Cooling:** Use aluminum heatsinks on CPU and memory
- **Case Selection:** Choose cases with good airflow

Power Management:

- **Power Supply:** Use official 5V/5A adapter
- **USB Power:** Avoid high-power USB devices without powered hub
- **Undervoltage Detection:** Monitor with `vcgencmd get_throttled`

Storage Optimization:

- **SD Card:** Use high-quality Class 10 or better
- **USB Storage:** Use USB 3.0 drives for better performance

- **NVMe:** Consider NVMe hat for maximum performance

Software Optimization:

Memory Management:

```
bash
```

```
# Configure memory split
```

```
sudo raspi-config
```

```
# Advanced Options → Memory Split → 64MB
```

```
# Optimize swap
```

```
sudo nano /etc/sysctl.conf
```

```
# Add: vm.swappiness=10
```

Network Optimization:

```
bash
```

```
# TCP optimization
```

```
sudo nano /etc/sysctl.conf
```

```
# Add network optimizations:
```

```
net.core.rmem_max = 16777216
```

```
net.core.wmem_max = 16777216
```

```
net.ipv4.tcp_rmem = 4096 87380 16777216
```

```
net.ipv4.tcp_wmem = 4096 65536 16777216
```

Security Considerations

System Security:

SSH Hardening:

bash

Configure SSH keys only

`sudo nano /etc/ssh/sshd_config`

Security settings:

PermitRootLogin no

PasswordAuthentication no

PubkeyAuthentication **yes**

MaxAuthTries **3**

ClientAliveInterval **300**

ClientAliveCountMax **2**

Firewall Configuration:

bash

Install and configure UFW

`sudo apt install ufw`

Default policies

`sudo ufw default deny incoming`

`sudo ufw default allow outgoing`

Allow SSH

`sudo ufw allow 2222/tcp`

Allow VNC

`sudo ufw allow 5901/tcp`

Enable firewall

`sudo ufw enable`

Network Security:

Network Monitoring:

```
bash
```

```
# Install network monitoring tools
```

```
sudo apt install ntopng
```

```
sudo apt install suricata
```

```
# Configure ntopng
```

```
sudo nano /etc/ntopng/ntopng.conf
```

```
# Add configuration:
```

```
-i=eth0
```

```
-P=/var/lib/ntopng/ntopng.pid
```

```
-d=/var/lib/ntopng
```

```
-w=3000
```

Maintenance and Updates

Regular Maintenance:

System Updates:

```
bash
```

```
#!/bin/bash
```

```
# ~/scripts/update.sh
```

```
# Update system
```

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
# Update firmware
```

```
sudo rpi-update
```

```
# Clean package cache
```

```
sudo apt autoremove -y
```

```
sudo apt autoclean
```

```
# Check system health
```

```
vcgencmd measure_temp
```

```
vcgencmd get_throttled
```

Log Management:

```
bash
```

```
# Configure log rotation
```

```
sudo nano /etc/logrotate.d/kali-logs
```

```
# Add log rotation rules:
```

```
/var/log/kali/*.log {  
    daily  
    rotate 7  
    compress  
    delaycompress  
    missingok  
    notifempty  
    create 644 root root  
}
```

Backup Strategy:

System Backup:

```
bash
```

```
#!/bin/bash
```

```
# ~/scripts/system-backup.sh
```

```
# Create system image
```

```
sudo dd if=/dev/mmcblk0 of=/mnt/persistence/backups/system-$(date +%Y%m%d).img bs=4M
```

```
# Create configuration backup
```

```
tar -czf /mnt/persistence/backups/config-$(date +%Y%m%d).tar.gz \  
    /etc/ \  
    /home/kali/ \  
    --exclude=/home/kali/.cache
```

Conclusion

This comprehensive guide provides everything needed to set up a powerful Kali Linux penetration testing platform on Raspberry Pi 5. The configuration includes:

- **Complete system setup** with persistence and optimization
- **Virtualization support** using Docker and QEMU/KVM
- **Network configuration** for access point and bridging

- **Hardware interface support** for GPIO, SPI, and I2C
- **Security hardening** and monitoring
- **Performance optimization** for sustained operation

The Raspberry Pi 5 running Kali Linux provides a portable, powerful platform for security testing, research, and education. With proper cooling and configuration, it can handle demanding penetration testing tasks while maintaining excellent power efficiency.

Remember to always use these tools ethically and legally, only on systems you own or have explicit permission to test.

For additional help and Pi-specific support, visit:

- Kali Linux Documentation: <https://www.kali.org/docs/>
- Raspberry Pi Foundation: <https://www.raspberrypi.org/>
- Kali Linux ARM: <https://www.kali.org/docs/arm/>